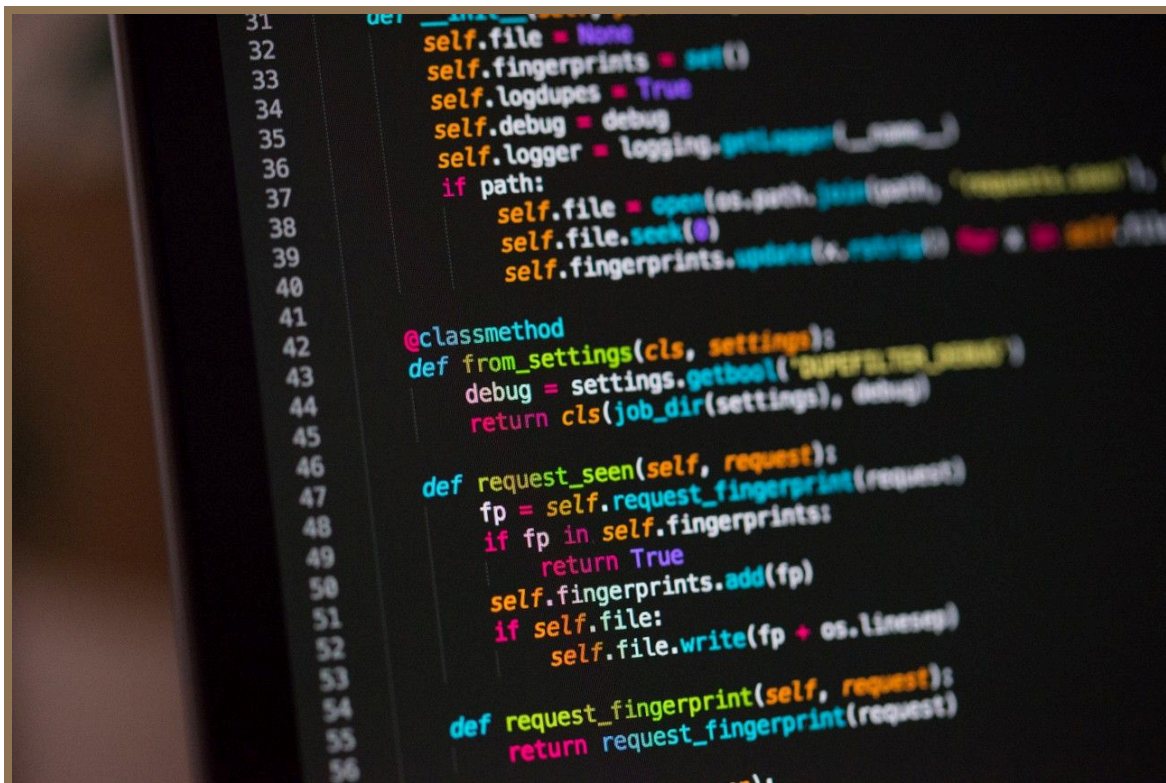


JUEGO - CÓDIGO SECRETO

PROYECTO 2



Profesor: Victor Racsó Galván Oyola

Integrantes:

- Santos Raymondi Emmanuel Isai - (20261PRECS01054)
- Ilicheva Svetlana - (20261PRECS01128)
- Tueros Hilares Amir - (20261PRECS01142)
- Villanueva Armas Alejandro Fabian - (20261PRECS01058)
- Zuñiga Canazas André Anthony - (20261PRECS01134)

Resumen

El presente informe detalla el desarrollo, la estructura y la implementación del juego "Código Secreto", un proyecto aplicativo en consola desarrollado en el lenguaje Python para el curso de Programación. En lo largo de este documento se definen los actores principales del sistema y se explica a profundidad el funcionamiento del código, el cual soporta modalidades de juego entre personas (automático y manual) y contra la computadora. Asimismo, se incluyen evidencias visuales mediante ejemplos de ejecución que demuestran el correcto flujo del programa, la gestión de errores, el almacenamiento persistente de partidas y la funcionalidad del algoritmo de verificación de consistencia. Finalmente, el documento cierra consolidando los resultados obtenidos mediante una serie de conclusiones y recomendaciones técnicas orientadas a la mejora continua del software desarrollado.

Introducción

Los juegos de deducción lógica han sido históricamente una excelente herramienta para estimular el pensamiento analítico, y llevarlos al ámbito computacional representa un desafío ideal para aplicar los fundamentos de la programación. En este contexto se enmarca el desarrollo de "Código Secreto", un juego interactivo donde los participantes compiten por descifrar el código oculto de su oponente (compuesto por tres dígitos únicos) a través de consultas estratégicas y deducciones basadas en las coincidencias obtenidas. Este proyecto va más allá de una simple simulación de adivinanza, ya que introduce retos computacionales interesantes, como la necesidad de diseñar un sistema robusto de "verificación de consistencia" que analiza en tiempo real el historial de respuestas en el modo manual para evitar contradicciones lógicas y detectar trampas. A través de este informe, se explorará la arquitectura de esta solución tecnológica en Python, demostrando cómo mediante el uso de módulos, manejo de archivos y estructuras de datos adecuadas, se logró construir una interfaz de terminal funcional que no solo permite el juego dinámico, sino que también gestiona estadísticas detalladas y el historial de los usuarios.

Definiciones:

Conceptos del Juego:

- **Código Secreto:** Un número compuesto por tres dígitos diferentes (del 0 al 9). No se permiten números repetidos (ejemplo: "123" es válido, pero "112" es inválido).
- **Consulta (o Intento):** El número de tres dígitos que un jugador ingresa en su turno para intentar adivinar el código de su oponente.
- **Acierto:** Un valor del 0 al 3 que indica cuántos números de la consulta están presentes en el código secreto, sin importar en qué posición se encuentren.
- **Lista de Códigos:** Es la lista inicial que contiene las 720 combinaciones válidas del juego. Funciona como la "memoria" de la computadora para ir descartando opciones turno a turno.
- **Verificación de Consistencia (e Inconsistencia):** Es una revisión automática del historial de jugadas. Si debido a pistas falsas o erróneas la lista de posibles códigos se queda completamente vacía (cero opciones), se declara una "inconsistencia" (contradicción) y la partida termina inmediatamente.

Conceptos de Programación (Python):

- **JSON:** Un formato de texto muy común para guardar información. En este proyecto se utiliza para guardar el historial y estadísticas de las partidas en un archivo llamado partida.json.
 - **Datetime:** Una herramienta de Python que permite capturar y guardar la fecha y hora exacta en la que finaliza una partida.
 - **Set (Conjunto):** Una colección de datos en Python que elimina automáticamente los elementos repetidos. Se utiliza para comprobar rápidamente que un código tenga tres dígitos distintos.
 - **Ljust:** Una función de texto en Python que alinea las palabras a la izquierda rellenando con espacios. Se usó para organizar los nombres de los jugadores y crear dos columnas visualmente separadas y ordenadas en la pantalla.
 - **F-strings:** Una forma práctica de dar formato a los textos en Python. Se usó para asegurar que los números mantengan siempre tres caracteres, añadiendo un cero a la izquierda si es necesario (por ejemplo, para que el número doce se guarde como "012" en lugar de "12").
 - **OS System (os.system):** Una instrucción que permite a Python ejecutar comandos directos en la computadora. Se usó para limpiar la pantalla de la consola (comando cls) en cada turno, logrando que el juego se vea como una aplicación limpia en lugar de un texto que baja infinitamente.
-

Modos:

- **Automático:** *Dos jugadores humanos compiten entre ellos, mientras el sistema calcula automáticamente los aciertos.*
- **Manual:** *Dos jugadores humanos compiten entre ellos, sin embargo ellos mismos informan los aciertos entre ellos sin que la computadora sepa sus códigos*
- **Computadora:** *Un jugador compite contra la computadora, que deduce el código del jugador usando la lógica*

Funcionamiento del Programa

El programa se organiza en dos módulos principales: uno de herramientas, que concentra las funciones de lógica del juego y de interfaz, y otro que implementa las modalidades de juego disponibles: automático, manual y contra la computadora, además de una opción para consultar el historial de partidas guardadas, las cuales hacen uso de dichas herramientas para su correcto funcionamiento. A continuación, se detalla el funcionamiento de las funciones del módulo [herramientas.py](#).

limpiar(): limpia la pantalla de la terminal, y se invoca antes de mostrar cualquier interfaz nueva, para evitar que se acumule información de pantallas anteriores.

menu_principal(): muestra el menú de inicio del programa, con las opciones de jugar entre personas, jugar contra la computadora, ver el historial de partidas, ver estadísticas o salir. Valida que la opción ingresada sea una de las disponibles y, en caso de un valor inválido, vuelve a solicitarla. Devuelve el número de la opción elegida por el usuario.

submenu_personas(): se muestra cuando el usuario elige jugar entre personas, y permite seleccionar entre el modo automático, el modo manual, o volver al menú principal. Al igual que el menú principal, valida la opción ingresada antes de devolverla.

generacion(): genera un código secreto de tres dígitos distintos entre 0 y 9. Genera cada cifra con `random.randint`, comparándola con las anteriores mediante bucles `while`, de modo que nunca se repita un dígito dentro del mismo código.

verificar(secreto, intento): recibe un código secreto y un intento, y devuelve la cantidad de dígitos del intento que están presentes en el secreto, sin considerar su posición. Esta función es utilizada tanto para evaluar las consultas de los jugadores como las de la computadora.

ganaroperder(x, y): compara dos códigos y devuelve True si son exactamente iguales, o False en caso contrario. Se utiliza para confirmar si un jugador adivinó el código completo cuando obtuvo 3 aciertos.

crear_lista(): genera la lista completa de los 720 códigos posibles, recorriendo los números del 0 al 999, formateándolos a 3 dígitos, y conservando únicamente aquellos cuyos tres dígitos son distintos entre sí (verificado mediante la longitud del conjunto `set(numero)`). Esta lista es la base tanto de la verificación de consistencia en el modo manual como del pool de candidatos en el modo contra la computadora.

reducir_lista(lista_actual, número, aciertos): recorre una lista de candidatos y conserva únicamente los códigos que, al ser comparados con un número dado mediante `verificar()`, producirían la misma cantidad de aciertos indicada. Es la función central del proceso de verificación de consistencia, y también permite construir la lista de candidatos de la computadora en sus distintos niveles de dificultad.

interfaz(nombre1, nombre2, consultas1, aciertos1, consultas2, aciertos2, turnoactual, iscomputer=False): limpia la pantalla y dibuja el estado actual de la partida, mostrando en columnas el historial de consultas y aciertos de ambos jugadores (o de un solo jugador, en el caso del modo contra la computadora, mediante el parámetro `iscomputer`), junto con el turno actual.

fin_de_partida(ganador, consultasganador, inconsistente=False): limpia la pantalla y muestra el resultado final de la partida. Si la partida terminó de forma válida, indica el nombre del ganador y la cantidad de turnos utilizados; si terminó por una inconsistencia detectada, informa dicha situación sin declarar ganador.

nombres(is2player): solicita por teclado el nombre de uno o dos jugadores, según corresponda al modo de juego, y los devuelve para ser usados a lo largo de la partida.

generar_archivo(jugador1, jugador2, modo, turnos, ganador, dificultad=None): arma un JSON con la información de la partida finalizada (jugadores, fecha y hora obtenida con `datetime.now()`, ganador, cantidad de turnos y modo de juego), agregando también la dificultad cuando corresponde al modo contra la computadora. Si ya existe un archivo `partida.json`, carga su contenido previo; en caso contrario, comienza un historial vacío. Luego añade la nueva partida a dicho historial y sobrescribe el archivo con la lista completa actualizada, permitiendo así conservar el registro de todas las partidas jugadas entre distintas ejecuciones del programa.

A continuación, se explica el funcionamiento del programa al iniciar

Al iniciar, el usuario elige el modo de juego deseado. **En el modo automático**, el sistema genera de forma aleatoria un código secreto para cada jugador. En cada turno, el jugador correspondiente ingresa una consulta de tres dígitos distintos, la cual es validada antes de ser procesada. El programa calcula la cantidad de aciertos comparando la consulta contra el código secreto del oponente, y actualiza la interfaz mostrando el historial de consultas y aciertos de ambos participantes. La partida finaliza cuando alguno de los 2 jugadores logra adivinar exactamente el código secreto de su oponente.

En el modo manual, los códigos secretos no son generados por el programa, sino acordados previamente entre los jugadores fuera del programa y de forma secreta. Cada jugador ingresa las consultas y el oponente declara manualmente la cantidad de aciertos obtenida. Para garantizar la validez de estas respuestas. El programa aplica un proceso de **verificación de consistencia**, y en caso de detectarse una inconsistencia, la partida finaliza automáticamente sin un ganador

En el modo contra la computadora, el jugador intenta adivinar un código secreto generado por el programa, al mismo tiempo que la computadora intenta adivinar el código secreto del jugador elegido fuera del programa a través de sus respuestas. La estrategia de la computadora varía según el nivel de dificultad seleccionado previamente: en la dificultad difícil, la máquina elige sus consultas a partir de una lista de candidatos reducida con el historial completo de aciertos recibidos; en dificultad normal, la reducción se realiza únicamente en base a la última respuesta obtenida; y en dificultad fácil, la computadora

selecciona sus consultas de manera aleatoria, sin considerar la información de los aciertos. En todos los casos, se evita repetir consultas previamente utilizadas.

Al finalizar cualquier partida con un resultado válido, el programa informa el nombre del ganador y la cantidad de turnos utilizados, y almacena un registro de la partida en un archivo en formato JSON. Este archivo acumula el historial completo de las partidas jugadas, incluyendo los nombres de los participantes, la fecha, el modo de juego, el ganador y la cantidad de turnos empleados, permitiendo así conservar dicha información entre distintas ejecuciones del programa.

Finalmente, el programa cuenta con una opción de historial, que permite visualizar en pantalla todas las partidas registradas en el archivo JSON, mostrando para cada una la información almacenada (jugadores, fecha, modo, ganador, turnos y, de corresponder, la dificultad).

Ejemplos de Ejecución

Captura1:Interfaz del Menú Principal (main.py)

```
=====
                        MENU PRINCIPAL
=====

1. Jugar entre personas
2. Jugar contra la computadora
3. Ver historial de partidas
4. Ver estadísticas
5. Salir

Ingrese el numero de la opcion: 1
```

Captura 2: Se elige el modo en el cual se desea jugar:

```
Ingresa el numero de la opcion: 1
=====
                        CODIGO SECRETO
=====

1. Modo Automatico
2. Modo Manual
3. Volver al menu principal

Ingresa el numero de la opcion: 1|
```

captura 3: Ingresa el nombre del jugador y se inicia la partida:

```
0742 804
Ingresa el nombre del jugador 1: player_1
Ingresa el nombre del jugador 2: player_2
=====
                        CODIGO SECRETO
=====

Jugador 1 (player_1)           Jugador 2 (player_2)
-----
Turno actual: player_1
Ingresa una consulta: |
```

Captura 4: En el transcurso de la partida se verán los intentos de cada jugador y sus aciertos:

```
=====
                        CODIGO SECRETO
=====

Jugador 1 (player_1)      Jugador 2 (player_2)
154 -> 1                  896 -> 0
745 -> 1                  236 -> 1
159 -> 0                  864 -> 1
728 -> 1                  827 -> 2
854 -> 2                  084 -> 1
408 -> 3

-----
Turno actual: player_2
Ingrese una consulta:
```

Captura 5: El player_2 ganó porque acertó los números en el orden correcto:

```
Turno actual: player_2
Ingrese una consulta: 742
=====
                        FIN DE LA PARTIDA
=====

Ganador: player_2
Turnos utilizados: 6

Presione ENTER para continuar...
```

Captura 4: Los datos de la partida se guardan en, (partida.json).

```
{"jugador1": "player_1", "jugador2": "player_2",  
  "fecha": "2026-07-05 19:47:44.750353",  
  "ganador": "player_2",  
  "turnos": 6,  
  "modo": "Automatico"}]
```

Conclusiones y Recomendaciones

Conclusiones:

- 1) Trabajar con el juego nos hizo ver que separar la lógica de la interfaz no es solo una buena práctica, sino algo que de verdad facilita el desarrollo del mismo juego. Al tener toda la parte de cálculo en distintos códigos, pudimos probar el funcionamiento del juego de forma más práctica y sencilla, además que la organización era más limpia permitiéndonos encontrar más fácilmente fallos en el código.
 - 2) Manejar los errores de entrada nos costó más tiempo del que esperábamos. Cada vez que ejecutamos el código nos encontrábamos con algún detalle que ajustar (dígitos repetidos, letras, campos vacíos), y eso nos enseñó que un programa pensado para que lo use otra persona necesita anticiparse a cosas que uno como programador nunca haría por accidente.
 - 3) Guardar las partidas en un archivo le dio sentido a todo lo demás. Sin la persistencia, las estadísticas y el historial no servirían de nada porque se borrarían al cerrar el programa. Ver que los datos seguían ahí al volver a abrirlo fue de las cosas más satisfactorias del proyecto.
-

-
- 4) Este proyecto nos hizo entender la importancia de la planificación antes de escribir un código, debido a que para cumplir con todos los requisitos necesitábamos entender a profundidad y tener un esquema con el plan que ejecutaría paso a paso nuestro código.

Recomendaciones:

- 1) Recomendamos usar .json para guardar los datos, algo que en nuestro caso resultó muy útil. JSON nos permitió guardar cada partida como un diccionario con toda su información requerida.
 - 2) Recomendamos escribir primero las funciones de la lógica y probarlas por separado antes de tocar la interfaz. Nosotros lo hicimos así y nos ahorró mucho tiempo. Si lo hubiéramos mezclado todo desde el inicio, habría sido mucho más difícil saber si un error venía del cálculo o de cómo se mostraba en pantalla.
 - 3) Recomendamos definir desde el inicio cómo se van a llamar las variables y las funciones antes de empezar a programar cada quien por su lado. En nuestro caso, ponerse de acuerdo en esto desde el principio nos hubiera ahorrado tiempo al momento de unir los módulos, porque terminamos teniendo que revisar y ajustar nombres para que todo calzara correctamente entre los archivos que trabajó cada integrante del grupo.
 - 4) Recomendamos, antes de entregar el proyecto, leer el código completo de principio a fin como si uno fuera el usuario que va a jugar por primera vez. A nosotros nos sirvió mucho hacer esto porque así encontramos los print() de prueba que se nos habían quedado mostrando información que no debía verse, algo que probablemente no hubiéramos notado si solo hubiéramos revisado que el programa se ejecuta sin errores.
-